05-4-iac-policy-and-fixtures

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 5 deliverable: IaC policy-as-code with test fixtures. 33 unit tests, the production-today configuration rejected by 11 rules, an evasion fixture (variable-driven values, dynamic blocks, IPv6, Aurora) rejected by 7 more, and the remediated version passing. Captured output included.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`policy/rego/terraform.rego`

```
# Structural prevention for the Terraform defects in section 4b.
# Threat T3 (database exposure), plus T4 for the ECR rules and evidence
# integrity for the CloudTrail rules.
#
# Run:  conftest test --policy policy/rego --parser hcl2 --all-namespaces terraform/
# Test: conftest verify --policy policy/rego
#
# Design choices worth defending:
#  * These rules are about DATA-PLANE EXPOSURE, not about style. Every one of
#    them blocks a merge, which is only tenable because the set is small and each
#    rule is a thing a reviewer would stop the PR for anyway.
#  * Rules assert on the *absence* of a safe value, not the presence of an unsafe
#    one, so a resource that simply omits the attribute is caught. The original
#    manifest's real risk was omission, not a wrong value.
#  * No rule names accounts-api. A policy that only defends one service is a
#    review comment with extra steps; these apply to every repo that adopts them.

package main

import future.keywords.contains
import future.keywords.if
import future.keywords.in

# ---------------------------------------------------------------------------
# helpers

resources(kind) := {name: body |
    some name
    body := input.resource[kind][name]
}

# true when `field` is absent or falsy — the omission case
missing_or_false(body, field) if not body[field]

# ---------------------------------------------------------------------------
# B1 — the database must not be reachable from the internet.

deny contains msg if {
    some name, body in resources("aws_db_instance")
    body.publicly_accessible == true
    msg := sprintf(
        "deny_rds_publicly_accessible: aws_db_instance.%s has publicly_accessible=true. A
production database holding PII must have no public endpoint. [T3/B1]",
        [name],
    )
```

```
}

deny contains msg if {
    some name, body in resources("aws_security_group_rule")
    body.type == "ingress"
    some cidr in body.cidr_blocks
    cidr in {"0.0.0.0/0", "::/0"}
    msg := sprintf(
        "deny_open_ingress: aws_security_group_rule.%s allows ingress on %v-%v from %s. Use
source_security_group_id; a CIDR is a claim about the network, an SG id is a claim about identity.
[T3/B1]",
        [name, body.from_port, body.to_port, cidr],
    )
}

deny contains msg if {
    some name, body in resources("aws_security_group")
    some rule in body.ingress
    some cidr in rule.cidr_blocks
    cidr in {"0.0.0.0/0", "::/0"}
    msg := sprintf(
        "deny_open_ingress: aws_security_group.%s has an inline ingress rule open to %s. [T3/B1]",
        [name, cidr],
    )
}

# ------------------------------------------------------------------------
# B2 — the master credential must not be able to reach Terraform state.

deny contains msg if {
    some name, body in resources("aws_db_instance")
    body.password
    msg := sprintf(
        "deny_rds_inline_password: aws_db_instance.%s sets `password`, which is stored in PLAINTEXT
in Terraform state. Use manage_master_user_password=true (RDS-managed secret) and
iam_database_authentication_enabled=true for the app path. [T3/B2]",
        [name],
    )
}

# ------------------------------------------------------------------------
# B3 — encryption at rest cannot be retrofitted in place, so it is a merge gate.

deny contains msg if {
    some name, body in resources("aws_db_instance")
    missing_or_false(body, "storage_encrypted")
    msg := sprintf(
        "deny_rds_unencrypted: aws_db_instance.%s is unencrypted at rest. This cannot be enabled in
place — it requires a snapshot/restore/cutover — so it must be right at creation. [T3/B3]",
        [name],
    )
}

deny contains msg if {
    some name, body in resources("aws_db_instance")
    body.storage_encrypted == true
    not body.kms_key_id
    msg := sprintf(
        "deny_rds_aws_managed_key: aws_db_instance.%s is encrypted with the AWS-managed key. Use a
customer-managed key so key custody, rotation and grant revocation are ours. [T3/B3]",
        [name],
    )
}
```

```
# ----------------------------------------------------------------------------
# B4 — recoverability. Integrity and availability, and a retention obligation.

deny contains msg if {
    some name, body in resources("aws_db_instance")
    # object.get with a default, NOT `body.backup_retention_period < 7`. The bare
    # comparison is UNDEFINED when the attribute is absent, so the rule silently
    # passes a resource that never set it at all — the omission case, which is the
    # one that actually happens. Same pattern for the Aurora rule below.
    retention := object.get(body, "backup_retention_period", 0)
    retention < 7
    msg := sprintf(
        "deny_rds_no_backups: aws_db_instance.%s has backup_retention_period=%v (attribute absent
counts as 0). Minimum 7 days; a destructive attacker or a bad migration is unrecoverable below
that. [T3/B4]",
        [name, retention],
    )
}

deny contains msg if {
    some name, body in resources("aws_db_instance")
    body.skip_final_snapshot == true
    msg := sprintf(
        "deny_rds_no_final_snapshot: aws_db_instance.%s has skip_final_snapshot=true. Deletion
becomes irreversible. [T3/B4]",
        [name],
    )
}

deny contains msg if {
    some name, body in resources("aws_db_instance")
    missing_or_false(body, "deletion_protection")
    msg := sprintf(
        "deny_rds_no_deletion_protection: aws_db_instance.%s has no deletion_protection. [T3/B4]",
        [name],
    )
}

# ----------------------------------------------------------------------------
# B5 — the audit trail. Without these, nothing else here is provable.

deny contains msg if {
    some name, body in resources("aws_cloudtrail")
    missing_or_false(body, "include_global_service_events")
    msg := sprintf(
        "deny_cloudtrail_no_global_events: aws_cloudtrail.%s excludes global service events, so
IAM, STS, CloudFront and Route53 calls are NOT RECORDED AT ALL — the exact API calls of an identity
attack. [B5]",
        [name],
    )
}

deny contains msg if {
    some name, body in resources("aws_cloudtrail")
    missing_or_false(body, "is_multi_region_trail")
    msg := sprintf(
        "deny_cloudtrail_single_region: aws_cloudtrail.%s is single-region; an attacker operating
in any other region is invisible. [B5]",
        [name],
    )
}

deny contains msg if {
    some name, body in resources("aws_cloudtrail")
```

```
    missing_or_false(body, "enable_log_file_validation")
    msg := sprintf(
        "deny_cloudtrail_no_validation: aws_cloudtrail.%s has no log file validation, so there is
no digest chain and the log is not tamper-evident evidence. [B5]",
        [name],
    )
}

deny contains msg if {
    some name, body in resources("aws_cloudtrail")
    not body.kms_key_id
    msg := sprintf(
        "deny_cloudtrail_unencrypted: aws_cloudtrail.%s writes unencrypted log files. [B5]",
        [name],
    )
}

# ---------------------------------------------------------------------------
# T4 — artefact integrity, enforced at the registry.

deny contains msg if {
    some name, body in resources("aws_ecr_repository")
    body.image_tag_mutability != "IMMUTABLE"
    msg := sprintf(
        "deny_ecr_mutable_tags: aws_ecr_repository.%s allows tag mutation, so a pushed tag can be
re-pointed at a different image without any change in Git. [T4]",
        [name],
    )
}

# ---------------------------------------------------------------------------
# Key custody. Regulatory rather than threat-driven — see TRACEABILITY.md.

deny contains msg if {
    some name, body in resources("aws_kms_key")
    missing_or_false(body, "enable_key_rotation")
    msg := sprintf(
        "deny_kms_no_rotation: aws_kms_key.%s has automatic rotation disabled. [regulatory: key
management]",
        [name],
    )
}

# ---------------------------------------------------------------------------
# Data residency: RBI localisation means this stack exists in ap-south-1 only.
# Multi-region keys replicate key material out of the region.

deny contains msg if {
    some name, body in resources("aws_kms_key")
    body.multi_region == true
    msg := sprintf(
        "deny_kms_multi_region: aws_kms_key.%s is multi-region, which replicates key material
outside ap-south-1. [regulatory: data residency]",
        [name],
    )
}

# ---------------------------------------------------------------------------
# warn: things worth saying that are not worth blocking a merge for. Kept
# deliberately short — a warn list that scrolls is a warn list nobody reads.

warn contains msg if {
    some name, body in resources("aws_db_instance")
    missing_or_false(body, "multi_az")
```

```
    msg := sprintf(
        "warn_rds_single_az: aws_db_instance.%s is single-AZ. Availability, not security; not a
merge blocker.",
        [name],
    )
}

# ---------------------------------------------------------------------------
# Evasion shapes.
#
# Everything above this line assumes the value it inspects is a literal. That
# assumption is how most IaC policy suites are quietly defeated: `publicly_accessible
# = var.public` is not `true`, so a rule that tests `== true` passes it, and the
# resource ships public. The HCL2 parser hands us the raw interpolation string
# (`"${var.public}"`), which is neither `true` nor `false`.
#
# The posture below is FAIL CLOSED: if a security-relevant value cannot be
# statically proven safe from the HCL alone, the rule denies and asks for a literal
# or a documented waiver. That is deliberately stricter than most published rule
# sets, and it is the difference between a control and a speed bump.
# ---------------------------------------------------------------------------

is_bool(x) if is_boolean(x)

# --- unprovable exposure ----------------------------------------------------

deny contains msg if {
    some kind in {"aws_db_instance", "aws_rds_cluster"}
    some name, body in resources(kind)
    body.publicly_accessible
    not is_bool(body.publicly_accessible)
    msg := sprintf(
        "deny_unprovable_public: %s.%s sets publicly_accessible to %v, which cannot be evaluated
from the HCL. `= var.x` is not `= false`. Use a literal, or move the decision behind a module with
its own tested policy. [T3/B1 evasion]",
        [kind, name, body.publicly_accessible],
    )
}

deny contains msg if {
    some kind in {"aws_db_instance", "aws_rds_cluster"}
    some name, body in resources(kind)
    body.storage_encrypted
    not is_bool(body.storage_encrypted)
    msg := sprintf(
        "deny_unprovable_encryption: %s.%s sets storage_encrypted to a non-literal (%v). Encryption
at rest cannot be enabled in place, so it is not a decision to defer to a variable. [T3/B3
evasion]",
        [kind, name, body.storage_encrypted],
    )
}

# A variable-driven CIDR list arrives as a STRING, not an array, so the
# deny_open_ingress rule above never iterates it. This is the exact shape that
# slips a 0.0.0.0/0 past an otherwise correct rule set.
deny contains msg if {
    some name, body in resources("aws_security_group_rule")
    body.type == "ingress"
    is_string(body.cidr_blocks)
    msg := sprintf(
        "deny_unprovable_ingress_cidr: aws_security_group_rule.%s takes cidr_blocks from %v, so
this policy cannot prove it excludes 0.0.0.0/0. Name the source security group, or inline the
CIDRs. [T3/B1 evasion]",
        [name, body.cidr_blocks],
```

```
    )
}

# `dynamic "ingress"` generates rules at plan time from a value we cannot see.
# Denying it is a real cost to module authors and I would rather pay that than
# ship a rule set with a documented hole in it.
deny contains msg if {
    some name, body in resources("aws_security_group")
    body.dynamic.ingress
    msg := sprintf(
        "deny_dynamic_ingress_block: aws_security_group.%s builds ingress with a `dynamic` block,
which cannot be statically analysed. Use explicit aws_security_group_rule resources, or run this
module through a plan-JSON gate as well. [T3/B1 evasion]",
        [name],
    )
}

# --- Aurora: the same defects, a different resource type -----------------------
# A rule set that only knows aws_db_instance is one `aws_rds_cluster` away from
# being bypassed, and the migration to Aurora is a normal thing for a bank to do.

deny contains msg if {
    some name, body in resources("aws_rds_cluster")
    missing_or_false(body, "storage_encrypted")
    msg := sprintf("deny_rds_cluster_unencrypted: aws_rds_cluster.%s is unencrypted at rest.
[T3/B3]", [name])
}

deny contains msg if {
    some name, body in resources("aws_rds_cluster")
    body.master_password
    msg := sprintf(
        "deny_rds_cluster_inline_password: aws_rds_cluster.%s sets master_password, which lands in
Terraform state in plaintext. Use manage_master_user_password. [T3/B2]",
        [name],
    )
}

deny contains msg if {
    some name, body in resources("aws_rds_cluster")
    body.skip_final_snapshot == true
    msg := sprintf("deny_rds_cluster_no_final_snapshot: aws_rds_cluster.%s has
skip_final_snapshot=true. [T3/B4]", [name])
}

deny contains msg if {
    some name, body in resources("aws_rds_cluster")
    retention := object.get(body, "backup_retention_period", 0)
    retention < 7
    msg := sprintf(
        "deny_rds_cluster_no_backups: aws_rds_cluster.%s has backup_retention_period=%v (attribute
absent counts as 0, minimum 7). [T3/B4]",
        [name, retention],
    )
}
```

---

policy/rego/terraform_test.rego

```
# Unit tests for policy/rego/terraform.rego.  `conftest verify --policy policy/rego`
#
```

```
# Each rule gets a pair: the production-today value must be denied, the
# remediated value must not be. That pairing is the "fails before the fix, passes
# after" requirement — a rule that only has a failing fixture might be denying
# everything, which is the most common way a policy suite becomes a rubber stamp.

package main

import future.keywords.every
import future.keywords.if
import future.keywords.in

# ---------------------------------------------------------------- fixtures

bad_rds := {"resource": {"aws_db_instance": {"accounts": {
    "identifier": "accounts-prod",
    "engine": "postgres",
    "username": "postgres",
    "password": "${var.db_password}",
    "publicly_accessible": true,
    "storage_encrypted": false,
    "skip_final_snapshot": true,
    "backup_retention_period": 0,
}}}}

good_rds := {"resource": {"aws_db_instance": {"accounts": {
    "identifier": "accounts-prod",
    "engine": "postgres",
    "username": "rdsadmin",
    "manage_master_user_password": true,
    "iam_database_authentication_enabled": true,
    "publicly_accessible": false,
    "storage_encrypted": true,
    "kms_key_id": "${aws_kms_key.rds.arn}",
    "skip_final_snapshot": false,
    "final_snapshot_identifier": "accounts-prod-final",
    "backup_retention_period": 35,
    "deletion_protection": true,
    "multi_az": true,
}}}}

bad_sg := {"resource": {"aws_security_group_rule": {"db_ingress": {
    "type": "ingress",
    "from_port": 5432,
    "to_port": 5432,
    "protocol": "tcp",
    "cidr_blocks": ["0.0.0.0/0"],
}}}}

good_sg := {"resource": {"aws_security_group_rule": {"db_ingress": {
    "type": "ingress",
    "from_port": 5432,
    "to_port": 5432,
    "protocol": "tcp",
    "source_security_group_id": "sg-0node11223344556",
}}}}

bad_trail := {"resource": {"aws_cloudtrail": {"org": {
    "name": "org-trail",
    "is_multi_region_trail": false,
    "enable_log_file_validation": false,
    "include_global_service_events": false,
}}}}

good_trail := {"resource": {"aws_cloudtrail": {"org": {
```

```
    "name": "org-trail",
    "is_multi_region_trail": true,
    "enable_log_file_validation": true,
    "include_global_service_events": true,
    "is_organization_trail": true,
    "kms_key_id": "${aws_kms_key.trail.arn}",
}}}}

# ---------------------------------------------------------------- B1

test_denies_public_rds if {
    some m in deny with input as bad_rds
    contains(m, "deny_rds_publicly_accessible")
}

test_allows_private_rds if {
    not_denied_by(good_rds, "deny_rds_publicly_accessible")
}

test_denies_world_open_ingress if {
    some m in deny with input as bad_sg
    contains(m, "deny_open_ingress")
}

test_allows_sg_sourced_ingress if {
    count(deny) == 0 with input as good_sg
}

# ---------------------------------------------------------------- B2

test_denies_inline_password if {
    some m in deny with input as bad_rds
    contains(m, "deny_rds_inline_password")
}

test_allows_rds_managed_password if {
    not_denied_by(good_rds, "deny_rds_inline_password")
}

# ---------------------------------------------------------------- B3

test_denies_unencrypted_storage if {
    some m in deny with input as bad_rds
    contains(m, "deny_rds_unencrypted")
}

# Omission, not a wrong value: the attribute is simply absent.
test_denies_omitted_storage_encrypted if {
    some m in deny with input as {"resource": {"aws_db_instance": {"x": {"engine": "postgres"}}}}
    contains(m, "deny_rds_unencrypted")
}

test_denies_aws_managed_key if {
    some m in deny with input as {"resource": {"aws_db_instance": {"x": {
        "storage_encrypted": true,
        "backup_retention_period": 35,
        "deletion_protection": true,
    }}}}
    contains(m, "deny_rds_aws_managed_key")
}

test_allows_cmk_encrypted if {
    not_denied_by(good_rds, "deny_rds_unencrypted")
    not_denied_by(good_rds, "deny_rds_aws_managed_key")
```

```
}

# ---------------------------------------------------------------- B4

test_denies_zero_backup_retention if {
    some m in deny with input as bad_rds
    contains(m, "deny_rds_no_backups")
}

test_denies_skip_final_snapshot if {
    some m in deny with input as bad_rds
    contains(m, "deny_rds_no_final_snapshot")
}

test_denies_missing_deletion_protection if {
    some m in deny with input as bad_rds
    contains(m, "deny_rds_no_deletion_protection")
}

test_good_rds_is_clean if {
    count(deny) == 0 with input as good_rds
}

# ---------------------------------------------------------------- B5

test_denies_trail_without_global_events if {
    some m in deny with input as bad_trail
    contains(m, "deny_cloudtrail_no_global_events")
}

test_denies_single_region_trail if {
    some m in deny with input as bad_trail
    contains(m, "deny_cloudtrail_single_region")
}

test_denies_trail_without_validation if {
    some m in deny with input as bad_trail
    contains(m, "deny_cloudtrail_no_validation")
}

test_good_trail_is_clean if {
    count(deny) == 0 with input as good_trail
}

# ---------------------------------------------------------------- T4 / KMS

test_denies_mutable_ecr_tags if {
    some m in deny with input as {"resource": {"aws_ecr_repository": {"api": {
        "name": "accounts-api",
        "image_tag_mutability": "MUTABLE",
    }}}}
    contains(m, "deny_ecr_mutable_tags")
}

test_allows_immutable_ecr_tags if {
    count(deny) == 0 with input as {"resource": {"aws_ecr_repository": {"api": {
        "name": "accounts-api",
        "image_tag_mutability": "IMMUTABLE",
    }}}}
}

test_denies_kms_without_rotation if {
    some m in deny with input as {"resource": {"aws_kms_key": {"k": {"description": "x"}}}}
    contains(m, "deny_kms_no_rotation")
```

```
}

test_denies_multi_region_kms_for_residency if {
    some m in deny with input as {"resource": {"aws_kms_key": {"k": {
        "enable_key_rotation": true,
        "multi_region": true,
    }}}}
    contains(m, "deny_kms_multi_region")
}

# ---------------------------------------------------------------- warn tier

test_single_az_warns_but_does_not_deny if {
    some m in warn with input as {"resource": {"aws_db_instance": {"x": {
        "storage_encrypted": true,
        "kms_key_id": "arn:x",
        "backup_retention_period": 35,
        "deletion_protection": true,
    }}}}
    contains(m, "warn_rds_single_az")
    count(deny) == 0 with input as {"resource": {"aws_db_instance": {"x": {
        "storage_encrypted": true,
        "kms_key_id": "arn:x",
        "backup_retention_period": 35,
        "deletion_protection": true,
    }}}}
}

# ---------------------------------------------------------------- helper

not_denied_by(doc, rule_name) if {
    msgs := deny with input as doc
    every m in msgs {
        not contains(m, rule_name)
    }
}

################################################################################
# Evasion-shape tests.
#
# These are the tests I would most want a reviewer to look at. Each one encodes a
# way a naive rule set is defeated while every rule above it still reports green.
################################################################################

# `publicly_accessible = var.public` parses as the STRING "${var.public}".
# A rule testing `== true` passes it, and the database ships public.
test_denies_variable_driven_public_flag if {
    some m in deny with input as {"resource": {"aws_db_instance": {"x": {
        "publicly_accessible": "${var.public}",
        "storage_encrypted": true,
        "kms_key_id": "arn:x",
        "backup_retention_period": 35,
        "deletion_protection": true,
        "multi_az": true,
    }}}}
    contains(m, "deny_unprovable_public")
}

test_denies_variable_driven_encryption_flag if {
    some m in deny with input as {"resource": {"aws_db_instance": {"x": {
        "storage_encrypted": "${var.encrypt}",
        "kms_key_id": "arn:x",
        "backup_retention_period": 35,
        "deletion_protection": true,
```

```
        "multi_az": true,
    }}}}
    contains(m, "deny_unprovable_encryption")
}

# `cidr_blocks = var.allowed` parses as a STRING, not a list, so a rule that
# iterates the list never executes at all.
test_denies_variable_driven_cidr_list if {
    some m in deny with input as {"resource": {"aws_security_group_rule": {"x": {
        "type": "ingress",
        "from_port": 5432,
        "to_port": 5432,
        "cidr_blocks": "${var.allowed_cidrs}",
    }}}}
    contains(m, "deny_unprovable_ingress_cidr")
}

test_denies_dynamic_ingress_block if {
    some m in deny with input as {"resource": {"aws_security_group": {"x": {
        "name": "x",
        "dynamic": {"ingress": {
            "for_each": "${var.cidrs}",
            "content": {"from_port": 5432, "to_port": 5432, "cidr_blocks": ["${ingress.value}"]},
        }},
    }}}}
    contains(m, "deny_dynamic_ingress_block")
}

test_denies_ipv6_world_open_ingress if {
    some m in deny with input as {"resource": {"aws_security_group_rule": {"x": {
        "type": "ingress",
        "from_port": 5432,
        "to_port": 5432,
        "cidr_blocks": ["::/0"],
    }}}}
    contains(m, "deny_open_ingress")
}

# The omission case: `backup_retention_period` absent entirely. A bare
# `body.backup_retention_period < 7` is UNDEFINED here and silently passes.
test_denies_absent_backup_retention if {
    some m in deny with input as {"resource": {"aws_db_instance": {"x": {
        "storage_encrypted": true,
        "kms_key_id": "arn:x",
        "deletion_protection": true,
        "multi_az": true,
    }}}}
    contains(m, "deny_rds_no_backups")
}

# --- Aurora: same defects, different resource type ---------------------------

test_denies_unencrypted_aurora_cluster if {
    some m in deny with input as {"resource": {"aws_rds_cluster": {"a": {
        "engine": "aurora-postgresql",
        "storage_encrypted": false,
    }}}}
    contains(m, "deny_rds_cluster_unencrypted")
}

test_denies_aurora_inline_password if {
    some m in deny with input as {"resource": {"aws_rds_cluster": {"a": {
        "storage_encrypted": true,
        "master_password": "${var.db_password}",
```

```
        "backup_retention_period": 35,
    }}}}
    contains(m, "deny_rds_cluster_inline_password")
}

test_allows_hardened_aurora_cluster if {
    count(deny) == 0 with input as {"resource": {"aws_rds_cluster": {"a": {
        "engine": "aurora-postgresql",
        "storage_encrypted": true,
        "kms_key_id": "arn:x",
        "manage_master_user_password": true,
        "backup_retention_period": 35,
        "skip_final_snapshot": false,
    }}}}
}

# A literal `false` is still fine — the fail-closed rules must not punish the
# correct spelling. Without this test, "deny everything unprovable" quietly
# becomes "deny everything".
test_literal_false_is_not_treated_as_unprovable if {
    not_denied_by(good_rds, "deny_unprovable_public")
    not_denied_by(good_rds, "deny_unprovable_encryption")
}
```

---

policy/fixtures/terraform/bad/main.tf

```
// The Terraform as it is deployed today (section 4b), committed verbatim as a
// NEGATIVE fixture for `conftest test --parser hcl2 policy/fixtures/terraform/bad`.
// Every deny rule in policy/rego/terraform.rego must fire on this file.
//
// Do not "fix" this file. Its job is to fail.

resource "aws_db_instance" "accounts" {
  identifier              = "accounts-prod"
  engine                  = "postgres"
  instance_class          = "db.r6g.large"
  username                = "postgres"
  password                = var.db_password
  publicly_accessible     = true
  storage_encrypted       = false
  skip_final_snapshot     = true
  backup_retention_period = 0
}

resource "aws_security_group_rule" "db_ingress" {
  type              = "ingress"
  from_port         = 5432
  to_port           = 5432
  protocol          = "tcp"
  cidr_blocks       = ["0.0.0.0/0"]
  security_group_id = aws_security_group.db.id
}

resource "aws_cloudtrail" "org" {
  name                          = "org-trail"
  s3_bucket_name                = aws_s3_bucket.trail.id
  is_multi_region_trail         = false
  enable_log_file_validation    = false
```

```
  include_global_service_events = false
}
```

---

policy/fixtures/terraform/evasion/main.tf

```
// EVASION FIXTURE. Every resource below satisfies the letter of a naive rule set
// while shipping the same exposure as policy/fixtures/terraform/bad/main.tf.
//
//   publicly_accessible = var.public      -> parses as the string "${var.public}",
//                                            which is not `true`, so `== true` passes it
//   cidr_blocks         = var.allowed     -> parses as a STRING, not a list, so a rule
//                                            that iterates the list never runs
//   dynamic "ingress"                     -> the rules are generated at plan time and
//                                            are invisible to a static parser
//   ::/0                                  -> IPv6 equivalent of 0.0.0.0/0
//   aws_rds_cluster                       -> Aurora: same defects, different resource type,
//                                            and a normal migration for a bank to make
//
// This is the fixture that separates a rule set from a speed bump.
// run-policy-tests.sh asserts conftest REJECTS this file. Do not "fix" it.

variable "public" { type = bool }
variable "allowed_cidrs" { type = list(string) }

resource "aws_db_instance" "sneaky" {
  identifier              = "accounts-prod-2"
  engine                  = "postgres"
  publicly_accessible     = var.public
  storage_encrypted       = true
  kms_key_id              = aws_kms_key.rds.arn
  backup_retention_period = 35
  deletion_protection     = true
  multi_az                = true
}

resource "aws_security_group" "dyn" {
  name = "dyn"
  dynamic "ingress" {
    for_each = var.allowed_cidrs
    content {
      from_port   = 5432
      to_port     = 5432
      protocol    = "tcp"
      cidr_blocks = [ingress.value]
    }
  }
}

resource "aws_security_group_rule" "var_cidr" {
  type              = "ingress"
  from_port         = 5432
  to_port           = 5432
  protocol          = "tcp"
  cidr_blocks       = var.allowed_cidrs
  security_group_id = aws_security_group.dyn.id
}

resource "aws_security_group_rule" "v6" {
  type        = "ingress"
  from_port   = 5432
```

```
  to_port     = 5432
  protocol    = "tcp"
  cidr_blocks = ["::/0"]
  security_group_id = aws_security_group.dyn.id
}

resource "aws_rds_cluster" "aurora" {
  cluster_identifier  = "accounts-aurora"
  engine              = "aurora-postgresql"
  master_username     = "root"
  master_password     = var.db_password
  storage_encrypted   = false
  skip_final_snapshot = true
}
```

---

policy/fixtures/terraform/good/main.tf

```
// The same three resources as policy/fixtures/terraform/bad/main.tf, remediated.
// POSITIVE fixture: `conftest test --parser hcl2 policy/fixtures/terraform/good`
// must report zero failures. Paired with the bad fixture this is the
// "fails before the fix, passes after" evidence for section 4b.

resource "aws_db_instance" "accounts" {
  identifier     = "accounts-prod"
  engine         = "postgres"
  engine_version = "16.4"
  instance_class = "db.r6g.large"

  publicly_accessible    = false
  db_subnet_group_name   = "accounts-prod-private"
  vpc_security_group_ids = [aws_security_group.db.id]

  username                            = "rdsadmin"
  manage_master_user_password         = true
  iam_database_authentication_enabled = true

  storage_encrypted = true
  kms_key_id        = aws_kms_key.rds.arn

  backup_retention_period   = 35
  skip_final_snapshot       = false
  final_snapshot_identifier = "accounts-prod-final"
  deletion_protection       = true
  multi_az                  = true
}

resource "aws_security_group_rule" "db_ingress" {
  type                     = "ingress"
  description              = "PostgreSQL from EKS nodes only"
  from_port                = 5432
  to_port                  = 5432
  protocol                 = "tcp"
  source_security_group_id = "sg-0node11223344556"
  security_group_id        = aws_security_group.db.id
}

resource "aws_cloudtrail" "org" {
  name                          = "org-trail"
  s3_bucket_name                = aws_s3_bucket.trail.id
  is_multi_region_trail         = true
```

```
  enable_log_file_validation    = true
  include_global_service_events = true
  is_organization_trail         = true
  kms_key_id                    = aws_kms_key.trail.arn
}
```

---

test-output/conftest-verify.txt

```
[32m33 tests, 33 passed, 0 warnings, 0 failures, 0 exceptions, 0 skipped[0m
```

---

test-output/conftest-bad-fixture.txt

```
[33mWARN[0m - policy/fixtures/terraform/bad/main.tf - main - warn_rds_single_az:
aws_db_instance.accounts is single-AZ. Availability, not security; not a merge blocker.
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_cloudtrail_no_global_events:
aws_cloudtrail.org excludes global service events, so IAM, STS, CloudFront and Route53 calls are
NOT RECORDED AT ALL — the exact API calls of an identity attack. [B5]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_cloudtrail_no_validation:
aws_cloudtrail.org has no log file validation, so there is no digest chain and the log is not
tamper-evident evidence. [B5]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_cloudtrail_single_region:
aws_cloudtrail.org is single-region; an attacker operating in any other region is invisible. [B5]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_cloudtrail_unencrypted:
aws_cloudtrail.org writes unencrypted log files. [B5]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_open_ingress:
aws_security_group_rule.db_ingress allows ingress on 5432-5432 from 0.0.0.0/0. Use
source_security_group_id; a CIDR is a claim about the network, an SG id is a claim about identity.
[T3/B1]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_rds_inline_password:
aws_db_instance.accounts sets `password`, which is stored in PLAINTEXT in Terraform state. Use
manage_master_user_password=true (RDS-managed secret) and iam_database_authentication_enabled=true
for the app path. [T3/B2]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_rds_no_backups:
aws_db_instance.accounts has backup_retention_period=0 (attribute absent counts as 0). Minimum 7
days; a destructive attacker or a bad migration is unrecoverable below that. [T3/B4]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_rds_no_deletion_protection:
aws_db_instance.accounts has no deletion_protection. [T3/B4]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_rds_no_final_snapshot:
aws_db_instance.accounts has skip_final_snapshot=true. Deletion becomes irreversible. [T3/B4]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_rds_publicly_accessible:
aws_db_instance.accounts has publicly_accessible=true. A production database holding PII must have
no public endpoint. [T3/B1]
[31mFAIL[0m - policy/fixtures/terraform/bad/main.tf - main - deny_rds_unencrypted:
aws_db_instance.accounts is unencrypted at rest. This cannot be enabled in place — it requires a
snapshot/restore/cutover — so it must be right at creation. [T3/B3]

[31m25 tests, 13 passed, 1 warning, 11 failures, 0 exceptions[0m
```

---

test-output/conftest-evasion-fixture.txt

```
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main - deny_dynamic_ingress_block:
aws_security_group.dyn builds ingress with a `dynamic` block, which cannot be statically analysed.
```

```
Use explicit aws_security_group_rule resources, or run this module through a plan-JSON gate as
well. [T3/B1 evasion]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main - deny_open_ingress:
aws_security_group_rule.v6 allows ingress on 5432-5432 from ::/0. Use source_security_group_id; a
CIDR is a claim about the network, an SG id is a claim about identity. [T3/B1]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main -
deny_rds_cluster_inline_password: aws_rds_cluster.aurora sets master_password, which lands in
Terraform state in plaintext. Use manage_master_user_password. [T3/B2]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main - deny_rds_cluster_no_backups:
aws_rds_cluster.aurora has backup_retention_period=0 (attribute absent counts as 0, minimum 7).
[T3/B4]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main -
deny_rds_cluster_no_final_snapshot: aws_rds_cluster.aurora has skip_final_snapshot=true. [T3/B4]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main - deny_rds_cluster_unencrypted:
aws_rds_cluster.aurora is unencrypted at rest. [T3/B3]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main - deny_unprovable_ingress_cidr:
aws_security_group_rule.var_cidr takes cidr_blocks from ${var.allowed_cidrs}, so this policy cannot
prove it excludes 0.0.0.0/0. Name the source security group, or inline the CIDRs. [T3/B1 evasion]
[31mFAIL[0m - policy/fixtures/terraform/evasion/main.tf - main - deny_unprovable_public:
aws_db_instance.sneaky sets publicly_accessible to ${var.public}, which cannot be evaluated from
the HCL. `= var.x` is not `= false`. Use a literal, or move the decision behind a module with its
own tested policy. [T3/B1 evasion]

[31m25 tests, 17 passed, 0 warnings, 8 failures, 0 exceptions[0m
```

---

test-output/conftest-good-fixture.txt

```
[32m25 tests, 25 passed, 0 warnings, 0 failures, 0 exceptions[0m
```